

SISMID Maps in R.Lab 1

Lance A. Waller

2026-07-14

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button (middle of options above) a document will be generated that includes both content as well as the output of any embedded R code chunks within the document.

Code chunks are bits of R code, usually to do a specific part of the problem. They appear in grey in the .Rmd file and you can run the chunk by itself by clicking the little green triangular “play” button in the top right of the grey chunk of code.

Mapping Lab 1

For this lab, will be making some basic maps using ggplot and ggplot2, the workhorse graphics plotting functions for the tidyverse. I’m drawing code and examples from two really nice examples available on the internet:

Dr. Emily Burchfield’s Maps in R (Emory Environmental Sciences) https://www.emilyburchfield.org/courses/eds/making_maps_in_r These examples work well but we don’t have access to the shapefile of Georgia interstate highway so I’ll leave out that part... (NOTE: A line of R code that starts with a # is commented out and ignored with you run the code).

The other source (referred to by Dr. Burchfield) is And “Making Maps with R” from Eric C. Anderson’s Reproducible Research Course for NOAA found at: <http://eriqande.github.io/rep-res-web/lectures/making-maps-with-R.html>

Let’s start with Eric Anderson’s code and see what’s involved. (NOTE: The code examples are his and I’ll put his text in quotes...I want to give proper credit.)

Installing packages

First, we need to install several libraries to R... these will contain the data and functions we will be using. You only need to run this code chunk the *first* time you run this code. If you have already installed these

```
# these are packages you will need, but probably already have.
# Don't bother installing if you already have them
install.packages(c("ggplot2", "devtools", "dplyr", "stringr"))

# some standard map packages.
install.packages(c("maps", "mapdata"))
```

Loading libraries

Next, we need to load the specific libraries of functions from within the packages. *You will need to do this each time your start R over, if you don't save your R workspace..* (NOTE: I never save my R workspace to

make sure I start clean each time.)

```
library(ggplot2)
#library(ggmap)
library(maps)
library(mapdata)
```

From Eric Anderson maps-package maps with ggplot

“The main players:

- The maps package contains a lot of outlines of continents, countries, states, and counties that have been with R for a long time.
- The mapdata package contains a few more, higher-resolution outlines.
- The maps package comes with a plotting function, but, we will opt to use ggplot2 to plot the maps in the maps package.
- Recall that ggplot2 operates on data frames. Therefore we need some way to translate the maps data into a data frame format the ggplot can use.”

“Maps in the maps package

- Package maps provides lots of different map outlines and points for cities, etc.
- Some examples: usa, nz, state, world, etc.”

ggplot2 provides the map_data() function.

- Think of it as a function that turns a series of points along an outline into a data frame of those points.

-Syntax: map_data(“name”) where “name” is a quoted string of the name of a map in the maps or mapdata package

Here we get a USA map from maps:

```
usa <- map_data("usa")
```

```
dim(usa)
```

```
## [1] 7243    6
```

```
#> [1] 7243    6
```

```
head(usa)
```

```
##      long      lat group order region subregion
## 1 -101.4078 29.74224    1     1   main      <NA>
## 2 -101.3906 29.74224    1     2   main      <NA>
## 3 -101.3620 29.65056    1     3   main      <NA>
## 4 -101.3505 29.63911    1     4   main      <NA>
## 5 -101.3219 29.63338    1     5   main      <NA>
## 6 -101.3047 29.64484    1     6   main      <NA>
```

```
#>      long      lat group order region subregion
#> 1 -101.4078 29.74224    1     1   main      <NA>
#> 2 -101.3906 29.74224    1     2   main      <NA>
#> 3 -101.3620 29.65056    1     3   main      <NA>
#> 4 -101.3505 29.63911    1     4   main      <NA>
#> 5 -101.3219 29.63338    1     5   main      <NA>
#> 6 -101.3047 29.64484    1     6   main      <NA>
```

```
tail(usa)
```

```
##           long      lat group order      region subregion
## 7247 -122.6187 48.37482    10  7247 whidbey island      <NA>
## 7248 -122.6359 48.35764    10  7248 whidbey island      <NA>
## 7249 -122.6703 48.31180    10  7249 whidbey island      <NA>
## 7250 -122.7218 48.23732    10  7250 whidbey island      <NA>
## 7251 -122.7104 48.21440    10  7251 whidbey island      <NA>
## 7252 -122.6703 48.17429    10  7252 whidbey island      <NA>
```

```
#>           long      lat group order      region subregion
#> 7247 -122.6187 48.37482    10  7247 whidbey island      <NA>
#> 7248 -122.6359 48.35764    10  7248 whidbey island      <NA>
#> 7249 -122.6703 48.31180    10  7249 whidbey island      <NA>
#> 7250 -122.7218 48.23732    10  7250 whidbey island      <NA>
#> 7251 -122.7104 48.21440    10  7251 whidbey island      <NA>
#> 7252 -122.6703 48.17429    10  7252 whidbey island      <NA>
```

The structure of those data frames

“These are pretty straightforward:

- long is longitude. Things to the west of the prime meridian are negative.
- lat is latitude.
- order. This just shows in which order ggplot should “connect the dots”
- region and subregion tell what region or subregion a set of points surrounds.
- group. This is very important! ggplot2’s functions can take a group argument which controls (amongst other things) whether adjacent points should be connected by lines. If they are in the same group, then they get connected, but if they are in different groups then they don’t.
- Essentially, having points in different groups means that ggplot “lifts the pen” when going between them.”

Plot the USA map

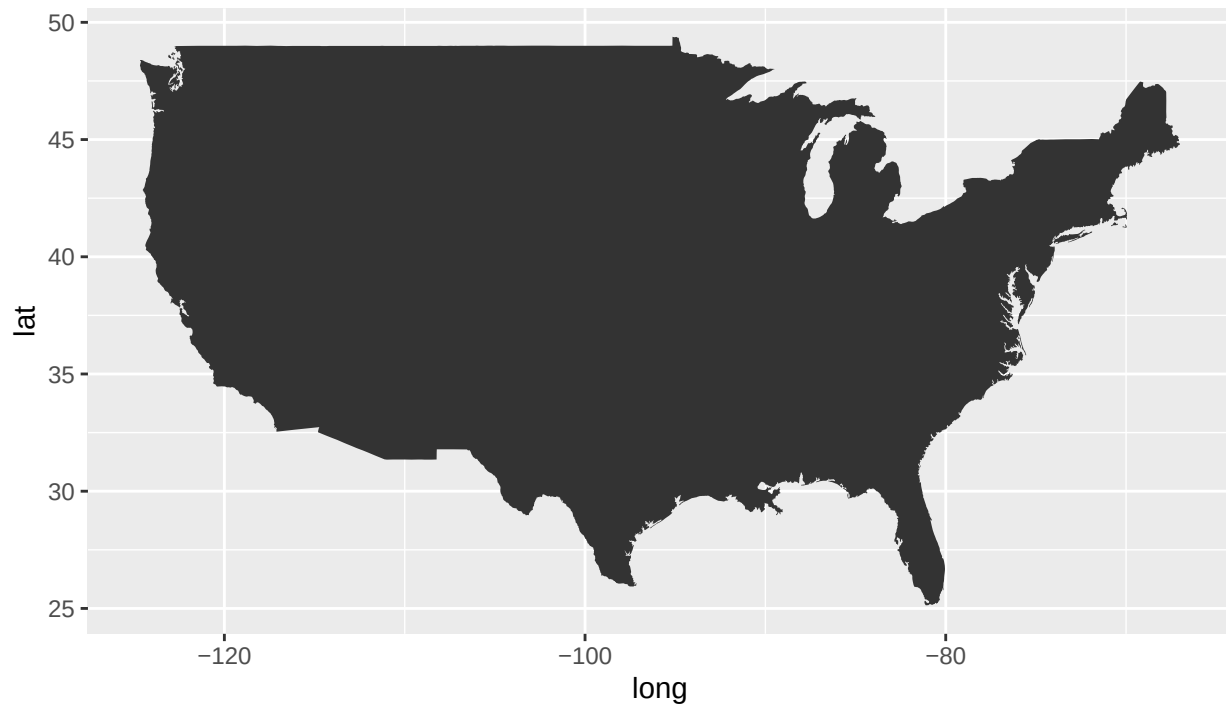
“Maps in this format can be plotted with the polygon geom. i.e. using `geom_polygon()`.”

- `geom_polygon()` drawn lines between points and “closes them up” (i.e. draws a line from the last point back to the first point)
- You have to map the group aesthetic to the group column
- Of course, `x = long` and `y = lat` are the other aesthetics.”

Simple black map

“By default, `geom_polygon()` draws with no line color, but with a black fill:”

```
usa <- map_data("usa") # we already did this, but we can do it again
ggplot() + geom_polygon(data = usa, aes(x=long, y = lat, group = group)) +
  coord_fixed(1.3)
```



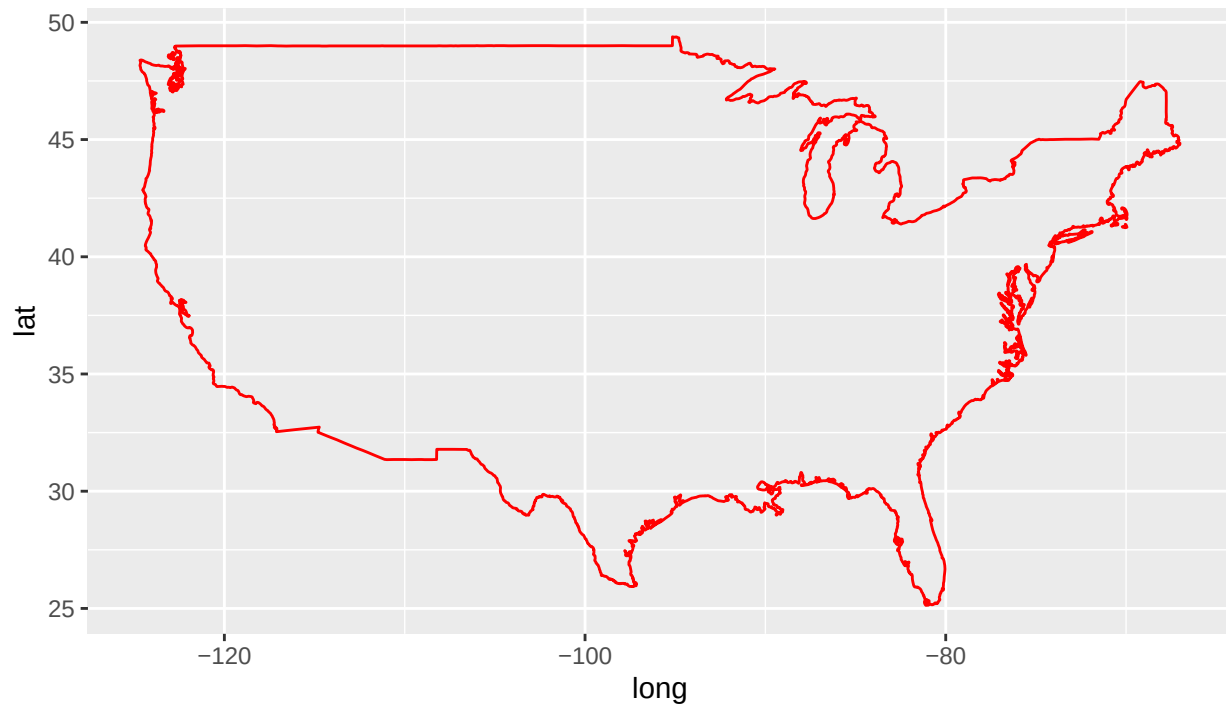
Eric: What is this `coord_fixed()`?

- This is very important when drawing maps.
- It fixes the relationship between one unit in the y direction and one unit in the x direction.
- Then, even if you change the outer dimensions of the plot (i.e. by changing the window size or the size of the pdf file you are saving it to (in ggsave for example)), the aspect ratio remains unchanged.
- In the above case, I decided that if every y unit was 1.3 times longer than an x unit, then the plot came out looking good.
- A different value might be needed closer to the poles.

Eric: Mess with line and fill colors

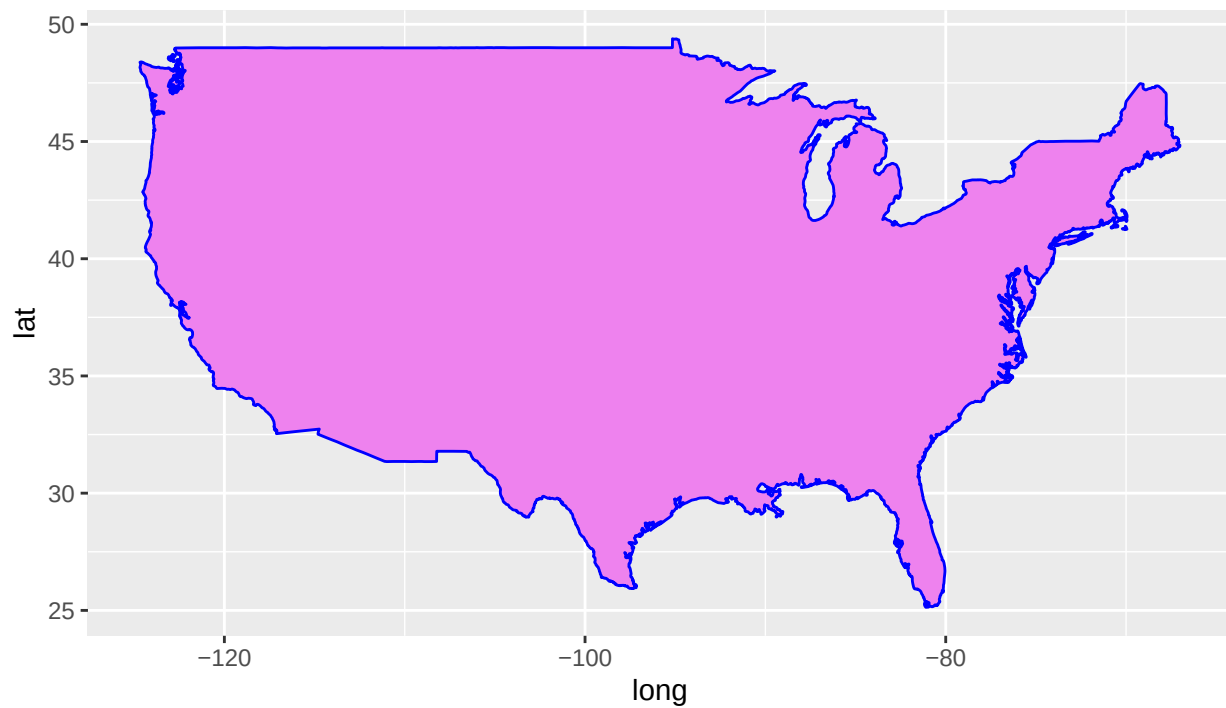
- Here is no fill, with a red line. Remember, fixed value of aesthetics go outside the aes function.

```
ggplot() +  
  geom_polygon(data = usa, aes(x=long, y = lat, group = group), fill = NA, color = "red") +  
  coord_fixed(1.3)
```



Eric: Violet fill, blue outline

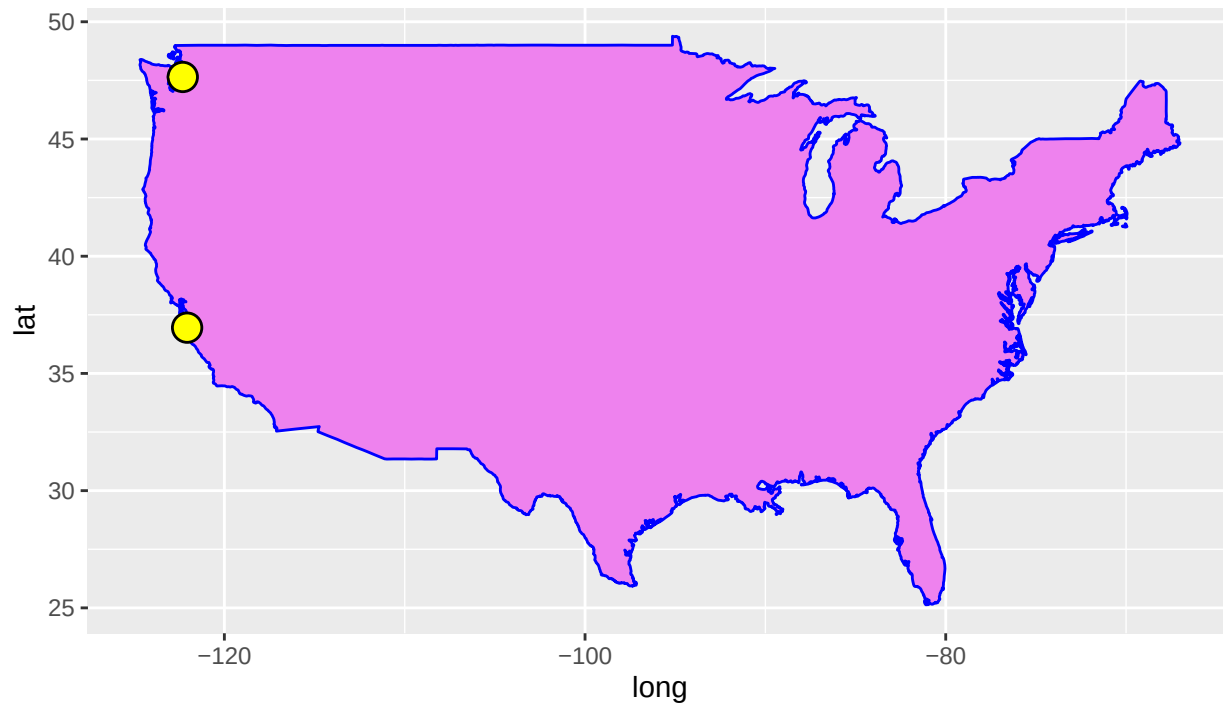
```
gg1 <- ggplot() +  
  geom_polygon(data = usa, aes(x=long, y = lat, group = group), fill = "violet", color = "blue") +  
  coord_fixed(1.3)  
gg1
```



Eric: Adding point so the map (given latitude and longitude)

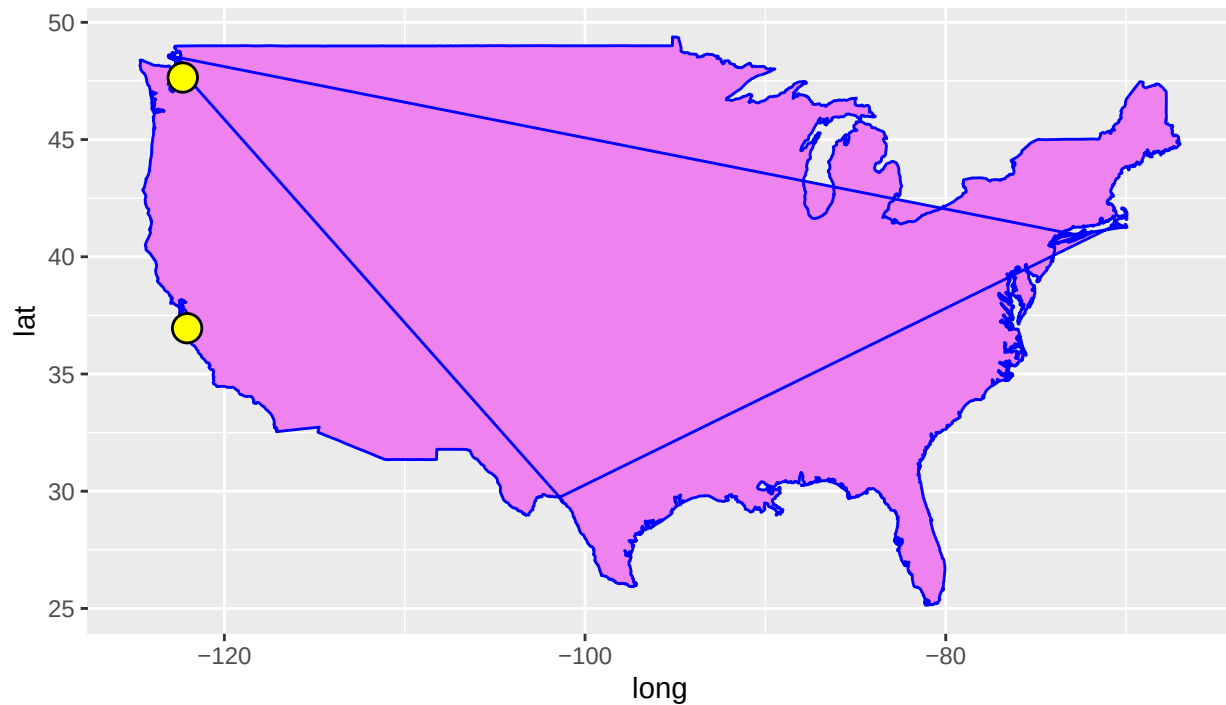
```
labs <- data.frame(
  long = c(-122.064873, -122.306417),
  lat = c(36.951968, 47.644855),
  names = c("SWFSC-FED", "NWFSC"),
  stringsAsFactors = FALSE
)

gg1 +
  geom_point(data = labs, aes(x = long, y = lat), color = "black", size = 5) +
  geom_point(data = labs, aes(x = long, y = lat), color = "yellow", size = 4)
```



What happens if we hadn't use group in defining our first plot (gg1)?

```
ggplot() +
  geom_polygon(data = usa, aes(x=long, y = lat), fill = "violet", color = "blue") +
  geom_point(data = labs, aes(x = long, y = lat), color = "black", size = 5) +
  geom_point(data = labs, aes(x = long, y = lat), color = "yellow", size = 4) +
  coord_fixed(1.3)
```



It didn't lift the pen!

Eric: State maps

Get the data:

```
states <- map_data("state")
dim(states)
```

```
## [1] 15537      6
```

```
#> [1] 15537      6
```

```
head(states)
```

```
##           long      lat group order  region subregion
## 1 -87.46201 30.38968     1     1  alabama    <NA>
## 2 -87.48493 30.37249     1     2  alabama    <NA>
## 3 -87.52503 30.37249     1     3  alabama    <NA>
## 4 -87.53076 30.33239     1     4  alabama    <NA>
## 5 -87.57087 30.32665     1     5  alabama    <NA>
## 6 -87.58806 30.32665     1     6  alabama    <NA>
```

```
#>           long      lat group order  region subregion
#> 1 -87.46201 30.38968     1     1  alabama    <NA>
#> 2 -87.48493 30.37249     1     2  alabama    <NA>
#> 3 -87.52503 30.37249     1     3  alabama    <NA>
#> 4 -87.53076 30.33239     1     4  alabama    <NA>
#> 5 -87.57087 30.32665     1     5  alabama    <NA>
#> 6 -87.58806 30.32665     1     6  alabama    <NA>
```

```
tail(states)
```

```
##           long      lat group order  region subregion
```

```
## 15594 -106.3295 41.00659 63 15594 wyoming <NA>
## 15595 -106.8566 41.01232 63 15595 wyoming <NA>
## 15596 -107.3093 41.01805 63 15596 wyoming <NA>
## 15597 -107.9223 41.01805 63 15597 wyoming <NA>
## 15598 -109.0568 40.98940 63 15598 wyoming <NA>
## 15599 -109.0511 40.99513 63 15599 wyoming <NA>
```

```
#>           long      lat group order  region subregion
#> 15594 -106.3295 41.00659 63 15594 wyoming <NA>
#> 15595 -106.8566 41.01232 63 15595 wyoming <NA>
#> 15596 -107.3093 41.01805 63 15596 wyoming <NA>
#> 15597 -107.9223 41.01805 63 15597 wyoming <NA>
#> 15598 -109.0568 40.98940 63 15598 wyoming <NA>
#> 15599 -109.0511 40.99513 63 15599 wyoming <NA>
```

Eric: Plot all the states, all colored a little differently

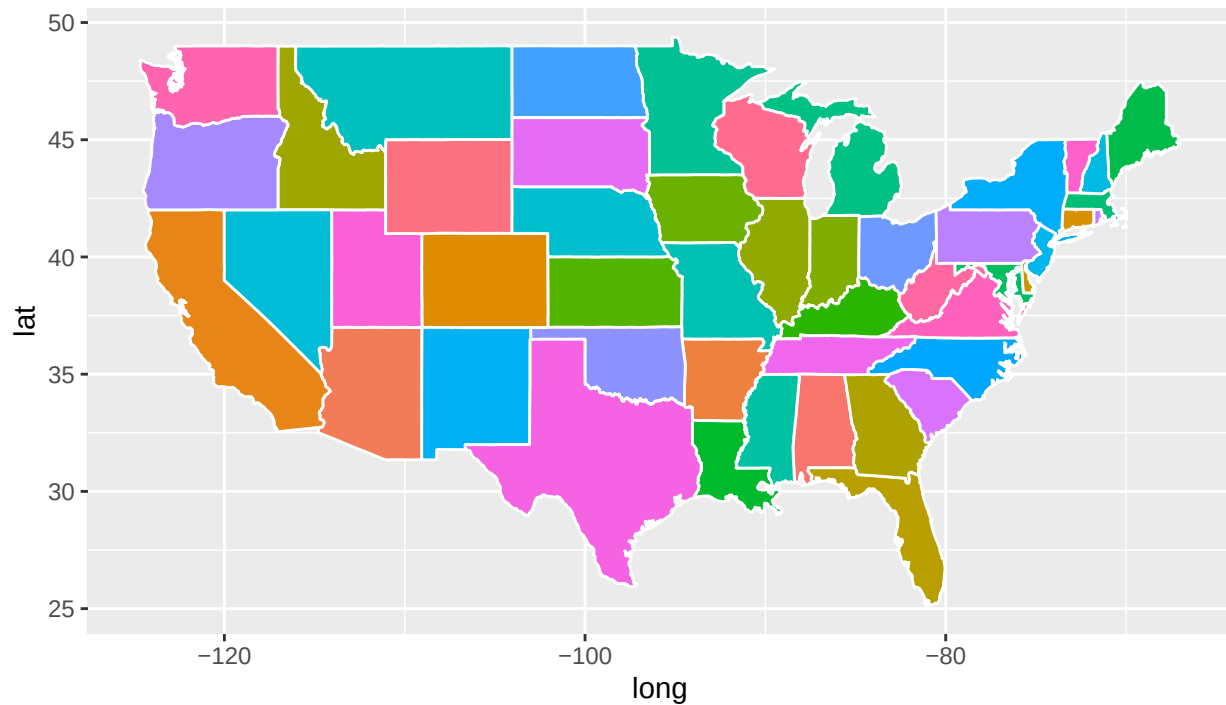
This is just like it is above, but we can map fill to region and make sure the the lines of state borders are white.

```
ggplot(data = states) +
  geom_polygon(aes(x = long, y = lat, fill = region, group = group), color = "white") +
  coord_fixed(1.3) +
  guides(fill=FALSE) # do this to leave off the color legend
```

```
## Warning: The `scale` argument of `guides()` cannot be `FALSE`. Use "none" instead as
## of ggplot2 3.3.4.
```

```
## This warning is displayed once per session.
```

```
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



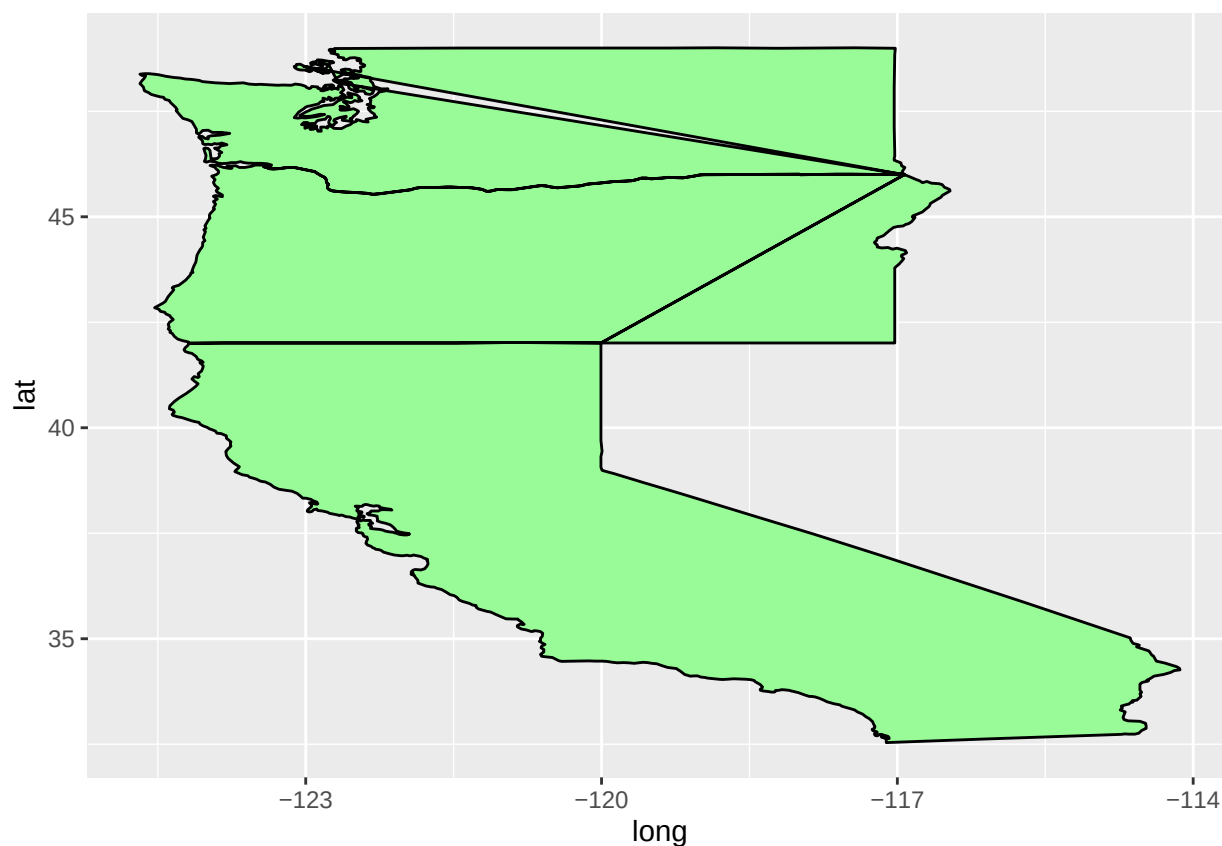
Eric: Plot just a subset of states in the contiguous 48:

Read about the subset command. It provides another way of subsetting data frames (sort of like using the `[]` operator with a logical vector).

We can use it to grab just CA, OR, and WA:

```
west_coast <- subset(states, region %in% c("california", "oregon", "washington"))

ggplot(data = west_coast) +
  geom_polygon(aes(x = long, y = lat), fill = "palegreen", color = "black")
```



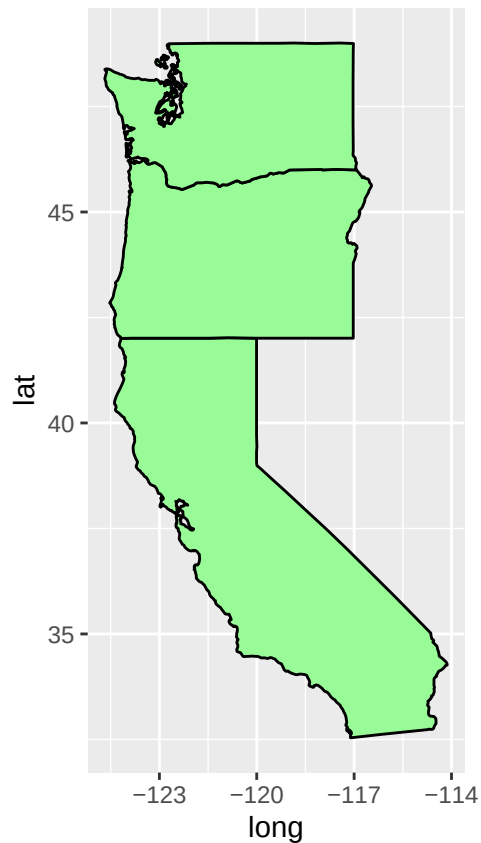
What happened?

Eric: I am just keeping people on their toes. What have we forgotten here?

- group
- coord_fixed()

Let's put those back in there:

```
ggplot(data = west_coast) +
  geom_polygon(aes(x = long, y = lat, group = group), fill = "palegreen", color = "black") +
  coord_fixed(1.3)
```



Eric: Phew! Lance: :)

Eric: Zoom in on California and look at counties

Get CA data

```
ca_df <- subset(states, region == "california")
```

```
head(ca_df)
```

```
##           long      lat group order    region subregion
## 667 -120.0060 42.00927     4   667  california      <NA>
## 668 -120.0060 41.20139     4   668  california      <NA>
## 669 -120.0060 39.70024     4   669  california      <NA>
## 670 -119.9946 39.44241     4   670  california      <NA>
## 671 -120.0060 39.31636     4   671  california      <NA>
## 672 -120.0060 39.16166     4   672  california      <NA>
```

```
#>           long      lat group order    region subregion
#> 667 -120.0060 42.00927     4   667  california      <NA>
#> 668 -120.0060 41.20139     4   668  california      <NA>
#> 669 -120.0060 39.70024     4   669  california      <NA>
#> 670 -119.9946 39.44241     4   670  california      <NA>
#> 671 -120.0060 39.31636     4   671  california      <NA>
#> 672 -120.0060 39.16166     4   672  california      <NA>
```

Eric: Let's get county lines

```
counties <- map_data("county")
ca_county <- subset(counties, region == "california")

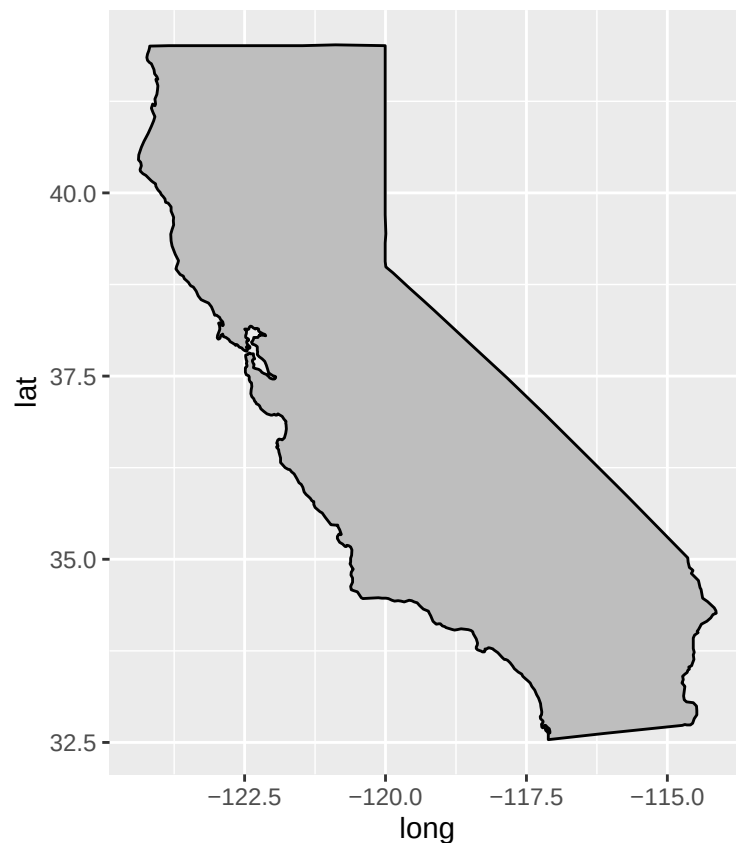
head(ca_county)
```

```
##           long      lat group order    region subregion
## 6965 -121.4785 37.48290   157  6965 california  alameda
## 6966 -121.5129 37.48290   157  6966 california  alameda
## 6967 -121.8853 37.48290   157  6967 california  alameda
## 6968 -121.8968 37.46571   157  6968 california  alameda
## 6969 -121.9254 37.45998   157  6969 california  alameda
## 6970 -121.9483 37.47717   157  6970 california  alameda
```

```
#>           long      lat group order    region subregion
#> 6965 -121.4785 37.48290   157  6965 california  alameda
#> 6966 -121.5129 37.48290   157  6966 california  alameda
#> 6967 -121.8853 37.48290   157  6967 california  alameda
#> 6968 -121.8968 37.46571   157  6968 california  alameda
#> 6969 -121.9254 37.45998   157  6969 california  alameda
#> 6970 -121.9483 37.47717   157  6970 california  alameda
```

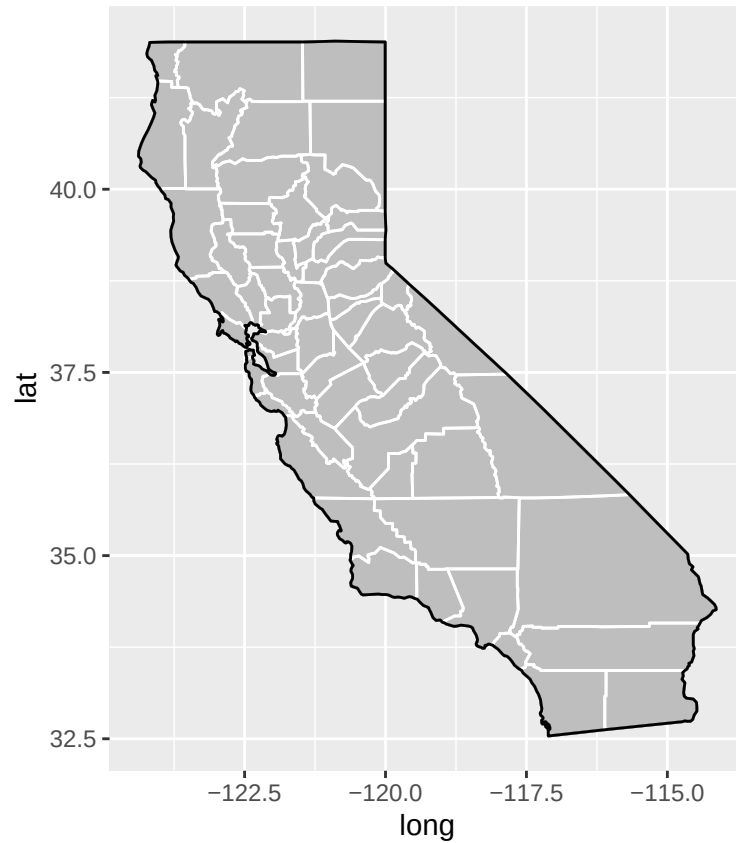
Eric: Plot the state first but let's ditch the axes gridlines, and gray background by using the super-wonderful theme: `nothing`.

```
ca_base <- ggplot(data = ca_df, mapping = aes(x = long, y = lat, group = group)) +
  coord_fixed(1.3) +
  geom_polygon(color = "black", fill = "gray")
ca_base #+ theme_nothing()
```



Eric: Now plot the county boundaries in white

```
ca_base + #theme_nothing() +  
  geom_polygon(data = ca_county, fill = NA, color = "white") +  
  geom_polygon(color = "black", fill = NA) # get the state border back on top
```



FOR YOU TO TRY: SISIMID participants, cut and paste code chunks above and try to update to plot the counties of the state of Georgia in the US ('ca' = 'California', 'ga' = 'Georgia')